Transformer 块

The Transformer Block

中文修订版·图表与公式随文排版

原作者：@techNmak·Understanding AI 系列·https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Transformer Block	Transformer 块	
Attention	注意力机制	Attention
Normalization	归一化	
Feed-forward Network	前馈网络	FFN
Feed-forward	前馈	
KV Cache	键值缓存	KV Cache
Models	模型	
SwiGLU	SwiGLU 门控激活	
RoPE	旋转位置嵌入	RoPE
MHA	多头注意力	MHA
MQA	多查询注意力	MQA
GQA	分组查询注意力	GQA
MLA	多头潜在注意力	MLA
Residual Stream	残差流	
Model	模型	
LayerNorm	层归一化	LayerNorm
RMSNorm	均方根归一化	RMSNorm
Output Projection	输出投影	
Incremental Decoding	增量解码	
Training	训练	
Prefill	预填充	
Matrix Multiplication	矩阵乘法	
Embeddings	嵌入向量	
Activation	激活	
Serving	推理服务	
Head Dimension	头维度	
Batch Size	批大小	

英文原词	中文译名	缩写／符号
Tensor	张量	
Query	查询	Q
Transformer	Transformer 模型	Transformer
Decoder	解码器	
Self-attention	自注意力	
Representation	表示	
Pre-norm	前置归一化	
Token	词元	
Gating	门控	
Heads	头	
Mental Model	理解框架	
Query Heads	查询头	
KV Heads	键值头	
Value Heads	值头	
Attention Head	注意力头	
Residual Connection	残差连接	
Cross-attention	交叉注意力	
Encoder	编码器	
Residual Path	残差路径	
Layer Normalization	层归一化	LayerNorm
Output Head	输出头	
Tensor Shape	张量形状	
Core	核心	
Vector	向量	
Evidence	证据	
Scaling	缩放	
Scaled Dot-product Attention	缩放点积注意力	SDPA
Matrix	矩阵	
Rotary Position Embedding	旋转位置嵌入	RoPE

英文原词	中文译名	缩写／符号
Token Embedding	词元嵌入	
Latent Representation	潜在表示	
Hidden State	隐藏状态	
Hidden Representation	隐藏表示	
Projection Matrix	投影矩阵	
Baseline	基线	
Rounding	舍入	
Decoder-only LLM	仅解码器大语言模型	
Queries	查询	
Values	值	
Keys	键	
Additive Mask	加性掩码	
Dot-product Attention	点积注意力	
Experts	专家	
Mixture-of-Experts	混合专家	MoE
Expert	专家	
Router	路由器	
Sparse Attention	稀疏注意力	
Global Attention	全局注意力	
Post-norm	后置归一化	
Causal Mask	因果掩码	
Decode	解码	
Tokens	词元	
Batching	批处理	
Kernel	内核	
Precision	精度	
Recomputation	重计算	
Forward Pass	前向传播	
Gradient	梯度	

英文原词	中文译名	缩写／符号
Inference	推理	
Dropout	随机失活	
Logit	逻辑值（归一化前的分数）	
Exact Attention	精确注意力	
Local Attention	局部注意力	
Routing	路由	
FlashAttention	FlashAttention（方法名保留）	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

Transformer 块 · 第 8 页

- 阅读导引 · 第 9 页

1 - 什么是变压器块 · 第 11 页

2 - 残余流 · 第 12 页

3 - 穿过块的形状 · 第 13 页

4 - 原 2017 年 LN 后区块 · 第 14 页

5 - 现代 PRE-LN 解码器（Decoder）模块 · 第 15 页

6 - 为什么残留添加很重要 · 第 16 页

7 - 层规范机制 · 第 17 页

8 - RMSnorm 机制 · 第 18 页

9 - 工作正常示例 · 第 19 页

10 - LN 前 VS LN 后 · 第 20 页

11 - 注意分支 · 第 21 页

12 - 绳索进入的位置 · 第 22 页

13 - 块内的 MHA、GQA、MQA 和 MLA · 第 23 页

14 - 输出投影（Output Projection）关闭注意力机制（Attention）分支 · 第 24 页

15 - MLP 是位置明智的 · 第 25 页

16 - 原始 RELU 前馈网络（Feed-forward Network） · 第 26 页

17 - 从 GELU 到门控 MLPS · 第 27 页

18 - 斯威格鲁机械 · 第 28 页

19 - 为什么门控 MLPS 使用不同的宽度 · 第 29 页

20 - 工作参数示例 · 第 30 页

21 - 加工形状追踪 · 第 31 页

22 - 串行与并行剩余块 · 第 32 页

23 - 编码器（Encoder）和解码器（Decoder）模块系列 · 第 33 页

24 - 因果掩盖受到关注 · 第 34 页

25 - MOE 取代 MLP 分支 · 第 35 页

26 - QK-NORM 不是块标准化 · 第 36 页

27 - GEMMA 2：一个有用的反例 · 第 37 页

28 - 训练（Training）、预填充（Prefill）和解码（Decode） · 第 38 页

29 - 键值缓存（KV Cache）属于注意力机制（Attention）状态 · 第 39 页

30 - 块级参数核算 · 第 40 页

31 - 计算直觉：注意力机制（Attention） VS MLP · 第 41 页

32 - 激活（Activation）内存和重新计算 · 第 42 页

33 - 辍学、偏见和其他小选择 · 第 43 页

34 - 透明的 Pytorch 块 · 第 44 页

35 - 常见的实施错误 · 第 45 页

36 - 常见的误解 · 第 46 页

37 - 完整的理解框架（Mental Model） · 第 47 页

38 - 紧凑架构比较 · 第 48 页

39 - 参考文献和扩展阅读 · 第 49 页

Transformer 块

对应原稿第 1 页

了解变压器块

从剩余流和对 **MLP** 的关注，

标准化和现代 **LLM** 块

面向人工智能工程师、开发人员和认真学习者的简短技术手册

@techNmak

手册09 - 理解AI系列

- 阅读导引

对应原稿第 2 页

本手册将澄清哪些问题

Transformer 块（Transformer Block）在图中看起来很简单：归一化（Normalization）、注意力机制（Attention）、前馈网络（Feed-forward Network）和两个残差加法。困难在于，现代语言模型（Models）几乎改变了该骨架周围的每个细节。有些在子层之前进行标准化，有些在子层之后进行标准化，有些则两者都进行。前馈（Feed-forward）分支可以使用 ReLU、GELU、SwiGLU、GeGLU 或稀疏 MoE。注意在解码期间可能会使用 MHA、GQA、MQA、MLA、RoPE、QK-Norm、本地窗口或 键值缓存（KV Cache）。

本手册将稳定结构与特定于体系结构的选择分开。到最后，您应该能够解释：

• 哪些信息流经残差流（Residual Stream）以及为什么每个子层必须返回模型（Model）宽度；

• Post-LN 和 Pre-LN 在代数上有何不同，而不仅仅是图表上的区别；

• LayerNorm 和RMSNorm 实际计算什么；

• 注意力机制（Attention）在块内开始和结束的位置，包括 RoPE、GQA 和输出投影（Output Projection）；

• 为什么MLP 是位置明智的，而整个块却不是；

• SwiGLU 如何改变前馈（Feed-forward）计算和参数计数；

• MoE 如何在不改变周围残差拓扑的情况下替换密集 MLP 分支；

• 训练（Training）、提示预填充（Prefill）和增量解码（Incremental Decoding）之间有何变化？

• 哪些细节足够通用以进行推理，哪些细节必须从模型（Model）配置中读取。

先备知识

对张量、矩阵乘法（Matrix Multiplication）、注意力机制（Attention）和神经网络层有基本的了解就足够了。早期关于 Attention、RoPE、KV Cache、MHA/MQA/GQA/MLA、MoE 和 Embeddings 的手册是有用的上下文，但这里的块是根据第一原理重建的。适用范围

这是一本关于 Transformer 块（Transformer Block）内部计算以及块如何堆叠的手册。它涉及内核、分布式执行、激活（Activation）检查点，并且仅在澄清块行为的地方提供推理服务（Serving）。这些系统主题值得单独处理。全文使用的符号

B= 批大小（Batch Size）； T= 序列长度； d= 模型（Model）/剩余宽度； H_q= 查询（Query）头计数； H_{kv}= 键/值头计数； d_h= 头维度（Head Dimension）； m= 前馈（Feed-forward）中间宽度。除非另有说明，张量（Tensor）形状是逻辑形状而不是所需的物理内存布局。

研究依据：[1]、[3]、[4]、[5]、[6]、[7]、[8]、[10]、[16]、[17]

1 - 什么是变压器块

对应原稿第 3 页

重复残差计算

Transformer 是通过堆叠重复更新隐藏向量序列的块来构建的。对于仅解码器（Decoder）的语言模型（Model），令

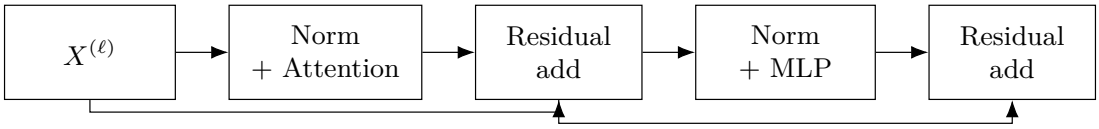
$$X^{(\ell)} \in \mathbb{R}^{B\times T\times d}$$

是进入 block ℓ.的残差表示（Representation） 常见的现代块有两个主要的可训练子层：自注意力（Self-attention）和前馈网络（Feed-forward Network）（MLP）。在预规范形式中，

$$U = X^{(\ell)} + \mathrm{Attn}(\mathrm{Norm}_1(X^{(\ell)})),$$

$$X^{(\ell+1)} = U + \mathrm{MLP}(\mathrm{Norm}_2(U)).$$

注意力机制（Attention）分支可以混合不同词元（Token）位置的信息。 MLP 独立作用于每个位置，但会变换其特征维度。剩余添加使两个子层保持连接到共享的 d维流。



+ 注意

规范

残差

添加

+ 多层感知机

规范

残差

添加

重要。“变压器块”并不是一种精确的实现。持久骨架是围绕注意力机制（Attention）和特征混合子层的重复残差更新。标准化布局、注意力机制类型、激活（Activation）、门控（Gating），甚至分支是否串行都可能有所不同。

研究依据：[1]、[4]、[8]、[10]

2 - 残余流

对应原稿第 4 页

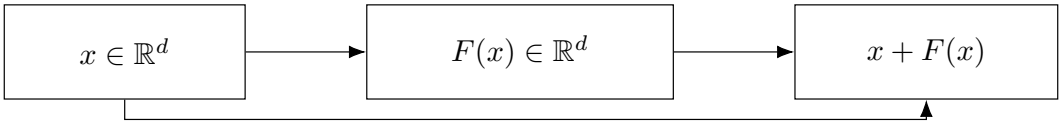
每个块读取和更新的状态

短语“残差流（Residual Stream）”是由残差连接通过堆栈承载的 d维向量序列的有用名称。如果子层计算更新 $F(x)$,，则残差规则为

$$y = x + F(x).$$

身份路径意味着旧的表示（Representation）仍然直接可用，而子层提供学习的修改。这个想法早于变形金刚：引入残差网络是为了通过学习相对于输入的残差函数，使非常深的网络更容易优化。 [2]

对于 Transformer 块（Transformer Block），注意力机制（Attention）和 MLP 都将更新写回到相同宽度的 d表示（Representation）中。这一要求解释了一个看起来任意的架构不变量：即使注意力机制暂时分裂成头或 MLP 扩展到更宽的隐藏维度，分支也必须在残差加法之前投影回 d。



理解框架（Mental Model）。块不会用“注意力机制（Attention）输出”和“MLP 输出”来替换表示（Representation）。每个分支都会提出对共享残差表示的更新。

研究依据：[1]、[2]

3 - 穿过块的形状

对应原稿第 5 页

在跟踪方程之前跟踪尺寸

对于具有剩余宽度 d,的解码器（Decoder）块

$$X \in \mathbb{R}^{B\times T\times d}.$$

标准化保留了这种形状。注意力机制（Attention）将最后一个维度投射到查询（Query）、键和值头（Value Heads）中。有 H_q查询头（Query Heads）、 H_{kv}KV头、及头维度 d_h,

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \qquad K, V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

注意力机制（Attention）之后，头部输出被组合并投射回

$$A \in \mathbb{R}^{B\times T\times d}.$$

然后，MLP 将每个词元（Token）表示（Representation）扩展为中间宽度 m并投影回来：

$$B \times T \times d \to B \times T \times m \to B \times T \times d.$$

舞台	逻辑形状	有什么变化?
剩余输入范数	$B \times T \times d$	nothing
	$B \times T \times d$	statistics/scale, not shape
Q	$B \times H_q \times T \times d_h$	split into query heads
K,V	$B \times H_{kv} \times T \times d_h$	KV heads may differ from Q
Attention output	$B \times T \times d$	heads recombined/projected
MLP hidden	$B \times T \times m$	feature expansion
Block output	$B \times T \times d$	back on residual stream

注意力机制（Attention）输出 MLP 隐藏块输出

没有统计/规模，没有形状分割成查询头（Query Heads） KV 头可能与 Q 头不同 重新组合/投影特征扩展回剩余流

重要。 MLP 宽度 m和注意头维度是临时内部宽度。余数加法要求分支输出返回 d.

研究依据：[1]、[11]、[20]

4 - 原 2017 年 LN 后区块

对应原稿第 6 页

先残差，后归一化（**Normalization**）

原始 Transformer 在每个子层周围应用残差连接（Residual Connection），然后对总和进行归一化。瓦斯瓦尼等人。将子层输出写为[1]

$$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x)).$$

对于编码器（Encoder）块，两个子层是自注意力（Self-attention）和位置前馈网络（Feed-forward Network）。解码器（Decoder）插入第三个子层用于编码器-解码器交叉注意。

对于两个子层的情况，简化的 Post-LN 块是

$$u = \mathrm{LN}(x + \mathrm{Attn}(x)),$$

$$y = \mathrm{LN}(u + \mathrm{FFN}(u)).$$

这个顺序很重要。标准化不仅仅是一个可以在不改变功能或优化行为的情况下移动的盒子。

原始模型（Model）还使用添加到嵌入向量（Embeddings）和 ReLU 前馈网络（Feed-forward Network）的正弦位置编码。这些选择在历史上很重要，但它们不是 Transformer 块（Transformer Block）抽象定义的一部分。

理解框架（Mental Model）。 Post-LN 意味着残差和在成为下一个子层或块的输入之前被标准化。

研究依据：[1]、[3]

5 - 现代 PRE-LN 解码器（Decoder）模块

对应原稿第 7 页

标准化分支输入，保持残差路径（Residual Path）直接

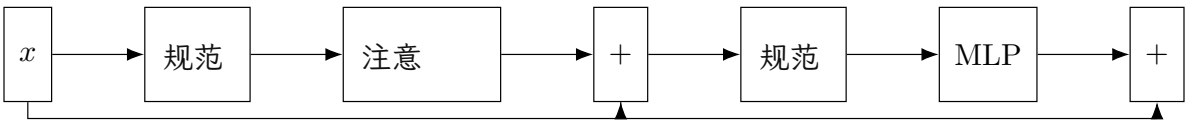
GPT-2 将层归一化（Layer Normalization）移至每个子块的输入，并在最后一个块之后添加最终归一化（Normalization）。 [4] GPT-3保留了GPT-2的预标准化架构。 [18] LLaMA 后来将 Pre-Norm 与 RMSNorm、SwiGLU 和 RoPE 结合起来。 [8]

常见的串行预范数块是

$$u = x + \mathrm{Attn}(\mathrm{Norm}_1(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}_2(u)).$$

最终模型（Model）级规范通常应用于语言模型输出头（Output Head）之前的最后一个块之后，但这是模型架构的选择，而不是每个单独块的一部分。



多层线性规划

重要。“现代变形金刚”并不意味着一种通用的预规范配方。例如，Gemma 2 使用前子层和后子层 RMSNorm。 [16]

研究依据：[4]、[8]、[16]、[18]

6 - 为什么残留添加很重要

对应原稿第 8 页

身份路径加上学习更新

考虑子层输出

$$y = x + F(x).$$

导数包含一个恒等项：

$$\frac{\partial y}{\partial x} = I + \frac{\partial F(x)}{\partial x}.$$

这并不意味着梯度在深层 Transformer 中永远不会消失或爆炸。归一化（Normalization）、初始化、参数规模、优化器动态以及许多块的组成仍然很重要。但直接身份路径是残差架构可以保留和细化表示的结构原因，而不是强迫每一层从头开始重新学习它们。 [2]

残余连接也会施加形状规则。如果 $x \in \mathbb{R}^d$，则 $F(x)$必须位于 $\mathbb{R}^d$的加法点处。这就是为什么具有多个头的注意力机制（Attention）分支以输出投影（Output Projection）结束，以及为什么扩展到宽度 m的 MLP 以向下投影结束的原因。

重要。残余连接不是串联。它是相同形状张量的元素加法。串联会改变宽度并需要不同的下游参数。

研究依据：[2]

7 - 层规范机制

对应原稿第 9 页

居中、缩放，然后应用学习到的仿射参数

对于一个词元（Token）向量 $x \in \mathbb{R}^d$,LayerNorm 计算标准化特征维度的统计数据。在常见的一维情况下，

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

$$; \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\mathrm{LN}(x)_i = \gamma_i \frac{x_i \quad \mu}{\sqrt{\sigma^2 \; + \epsilon}} + \beta_i.$$

最初的 LayerNorm 论文介绍了标准化后每个神经元的增益和偏差。 [3] 当前的 PyTorch LayerNorm 默认情况下同样支持可学习的元素缩放和偏差。 [19]

对于序列模型（Models），LayerNorm 通常在其特征维度上独立应用于每个词元（Token），而不是跨批次示例，也不跨序列位置。

理解框架（Mental Model）。 LayerNorm 更改每个词元（Token）表示（Representation）的坐标统计数据，但保持张量形状（Tensor Shape）不变。

研究依据：[3]、[19]

8 - RMSnorm 机制

对应原稿第 10 页

按均方根缩放而不减去平均值

RMSNorm 删除了 LayerNorm 的显式重新居中步骤。对于 $x \in \mathbb{R}^d$,

$$\mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{d}\sum_{i=1}^{d} x_i^2},$$

$$\mathrm{RMSNorm}(x)_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

张和 Sennrich 提出了 RMSNorm，其假设是 LayerNorm 的重新居中不变性在许多设置中是可有可无的。[6] 当前 PyTorch 的 RMSNorm 遵循相同的核心操作并保留输入形状。 [20]

	LayerNorm	RMSNorm
Subtract mean	yes	no
Scale statistic	standard deviation	root mean square
Learned scale	usually yes	usually yes
Learned bias	commonly available	not part of core RMSNorm formula
Shape change	none	none

减去平均值 尺度统计 学习尺度 学习偏差 形状变化

普遍可用

标准差

LayerNorm

通常是的

无

是的

不是核心 RMSNorm 公式的一部分

均方根

RMSNorm

通常是的

无

LLaMA 在 Pre-Norm Transformer 中使用 RMSNorm，Qwen3 同样报告具有预归一化（Normalization）的 RMSNorm。 [8,17]

研究依据：[6]、[8]、[17]、[20]

9 - 工作正常示例

对应原稿第 11 页

LayerNorm 和 RMSNorm 对同一向量（Vector）执行不同的操作

忽略习得的增益/偏差并采取

$$x = [1, 2, 3, 4].$$

对于 LayerNorm，

$$\mu = 2.5, \qquad \sigma^2 = 1.25, \qquad \sigma \approx 1.1180.$$

为了可读性忽略 ϵ，

$$\mathrm{LN}(x) \approx [-1.3416, -0.4472, 0.4472, 1.3416].$$

归一化向量（Vector）以零为中心。

对于 RMSNorm，

$$\frac{\sqrt{1^2 + 2^2 + 3^2 + 4^2}}{4}$$

$$\mathrm{RMS}(x) = \sqrt{\frac{1^2 + 2^2 + 3^2 + 4^2}{4}} = \sqrt{7.5} \approx 2.7386,$$

so

$$\mathrm{RMSNorm}(x) \approx [0.3651, 0.7303, 1.0954, 1.4606].$$

它的平均值不会被强制为零，因为该操作永远不会减去 μ.

重要。 LayerNorm 和 RMSNorm 不是具有不同名称的可互换公式。模型（Model）检查点是使用特定的标准化规则和参数化进行训练的。

研究依据：[3]、[6]

10 - LN 前 VS LN 后

对应原稿第 12 页

相同的成分，不同的优化路径

代数区别是紧凑的：

逻辑节点后：

$$y = \mathrm{Norm}(x + F(x)),$$

$$y = x + F(\mathrm{Norm}(x)).$$

预 LN：

在 Post-LN 中，身份路径在到达下一个块之前经过标准化。在 Pre-LN 中，残余流具有跨子层的直接附加路径。

熊等人。用平均场理论分析了这两个位置。在初始化时，他们发现原始 Post-LN Transformer 中的输出层附近存在较大的预期梯度，从而激发了学习率预热，而 Pre-LN 公式在他们的分析和实验中具有更好的梯度表现。 [5]

该结果是优化行为的证据（Evidence），而不是预逻辑网络总是更好的普遍说法。后来的架构以不同的方式重新审视了归一化（Normalization）和残差缩放（Scaling）。例如，DeepNorm 修改残差连接和初始化以稳定非常深的 Transformer。 [21]

理解框架（Mental Model）。归一化（Normalization）放置改变了梯度遍历的计算图。这不是外观上的重新排列。

研究依据：[1]、[4]、[5]、[21]

11 - 注意分支

对应原稿第 13 页

区块的词元（**Token**）混合子层

对于隐藏状态 $X \in \mathbb{R}^{B\times T\times d}$,，自注意力（Self-attention）首先形成学习投影。在传统的多头配方中，

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

重塑为头部后，缩放点积注意力（Scaled Dot-product Attention）计算

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

其中 M表示掩蔽或注意偏差。然后，头输出被重新组合并通过 W_O投影回剩余添加之前的宽度 d。 [1]

注意力机制（Attention）分支是词元（Token）位置可以交换信息的地方。这使得它与 MLP 分支有根本的不同，MLP 分支独立地将相同的特征变换应用于每个 词元 位置。

重要。残差分支接收注意力机制（Attention）输出投影（Output Projection），而不是注意力机制概率矩阵（Matrix）。注意力机制权重是中间值。

研究依据：[1]、[22]

12 - 绳索进入的位置

对应原稿第 14 页

位置修改attention内的Q/K兼容性

现代仅解码器（Decoder） LLMs 通常使用旋转位置嵌入（Rotary Position Embedding） (RoPE)。 RoPE 不会通过向 X.添加位置向量（Vector）来改变残差流（Residual Stream），相反，标准 RoPE 在计算点积之前对查询（Query）和关键组件应用位置相关的旋转。

[12]

因此，在预范数块中，概念顺序是

$X \rightarrow \text{Norm}(X) \rightarrow Q, K, V \rightarrow \text{RoPE}(Q, K) \rightarrow \text{Attention.})$

LLaMA 在每一层明确用 RoPE 替换绝对位置嵌入向量（Embeddings）。 [8]

这种区别很重要，因为它将三个经常折叠在一起的对象分开：

• 残差表示 X;

• 从 X;导出的学习的Q/K/V 投影

• 应用于Q 和K 的位置相关变换。

重要。 RoPE 并未“添加到 LLaMA 风格的区块中的词元嵌入（Token Embedding）中”。它改变了注意力机制（Attention）内部的 Q/K 交互。

研究依据：[8]、[12]

13 - 块内的 MHA、GQA、MQA 和 MLA

对应原稿第 15 页

当注意力机制（**Attention**）表示（**Representation**）发生变化时，块骨架保持不变

残差拓扑不会告诉您存在多少个键/值头（Value Heads）或在解码期间缓存了哪些表示（Representation）。

使用 H_q查询头（Query Heads）和 H_{kv}键/值头（Value Heads），

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \quad K,V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

MHA一般使用 $H_{kv}= H_q$.MQA使用1个KV头。 GQA 使用中间编号。 [11] 例如，Qwen3 在其密集模型（Models）中报告了 GQA。 [17]

MLA 采用不同的路线：DeepSeek-V2 将键/值信息压缩为低维潜在表示（Latent Representation），并单独处理 RoPE 敏感组件。 [14] 然而，在注意力机制（Attention）计算之后，分支仍然向残差流（Residual Stream）返回宽度d更新。

理解框架（Mental Model）。 MHA/GQA/MQA/MLA 更改注意力机制（Attention）分支的内部结构。它们不会取代块周围的范数残差 MLP 结构。

研究依据：[11]、[14]、[17]

14 - 输出投影（Output Projection）关闭注意力机制（Attention）分支

对应原稿第 16 页

打印头必须返回到剩余宽度

假设每个 H_q查询头（Query Heads）生成一个维度为 d_h.的向量（Vector） 连接头输出生成宽度的表示（Representation） $H_q d_h$.在常见情况下 $H_q d_h = d$,连接的张量（Tensor）已经具有宽度 d,但它仍然通过学习的输出投影（Output Projection）传递

$$W_O \in \mathbb{R}^{d \times d}.$$

然后分支返回

$$A = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_{H_q}) W_O$$

与 shape $B \times T \times d$,制作

$$x + A$$

明确的。 [1]

输出投影（Output Projection）不仅仅是重塑。它是通过学习将头部输出混合回模型（Model）的残差特征空间。

重要。只计算 W_Q, W_K, W_V就漏掉了标准MHA: W_O.中四大投影矩阵之一

研究依据：[1]

15 - MLP 是位置明智的

对应原稿第 17 页

没有词元（**Token**）混合的特征混合

原始 Transformer 将其第二个子层称为位置前馈网络（Feed-forward Network），因为相同的非线性函数独立应用于每个序列位置。[1]

如果 $X \in \mathbb{R}^{B\times T\times d}$ 和

$$f : \mathbb{R}^d \to \mathbb{R}^d,$$

然后

$$Y_{b,t,:} = f(X_{b,t,:})$$

对于每个批处理元素 b 和词元（Token）位置 $t.$，MLP不直接读取相邻词元位置。所有可用的跨词元信息必须已经存在于该词元的当前隐藏状态（Hidden State）中，通常是在注意力机制（Attention）在序列中混合信息之后。

这为区块提供了有用的分工：

• 注意力机制（Attention）将不同位置的信息混合在一起；

• MLP 在每个位置的特征向量（Vector）内转换信息。

理解框架（Mental Model）。“仓位”是指跨词元（Token）仓位共享功能、独立应用。这并不意味着 MLP 不知道上下文：它的输入状态已经可以包含由早期注意力机制（Attention）混合的上下文。

研究依据：[1]

16 - 原始 RELU 前馈网络（Feed-forward Network）

对应原稿第 18 页

扩展、激活、投影回

2017 Transformer 使用两个线性变换，它们之间有 ReLU：[1]

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

具有剩余宽度 d和前馈（Feed-forward）宽度 m,

$$W_1 \in \mathbb{R}^{d\times m}, \qquad W_2 \in \mathbb{R}^{m\times d}.$$

原来的基础Transformer使用了 $d = 512$和 $m = 2048$,一个4×扩展。

忽略偏差，这个密集 FFN 的参数计数为

$$P_{\mathrm{FFN}} = 2dm.$$

对于 $m = 4d$,

$$P_{\mathrm{FFN}} = 8d^2.$$

这就是为什么前馈（Feed-forward）分支可以包含密集 Transformer 的大部分参数，即使它从不混合词元（Token）位置。

重要。$4d$比率是原始架构的设计选择，而不是 Transformers 的数学要求。现代门控 MLP 通常使用不同的扩展比。

研究依据：[1]

17 - 从 GELU 到门控 MLPS

对应原稿第 19 页

非线性是架构的一部分

Transformer MLP 并未保持固定在 ReLU。后来的 Transformer 语言模型（Models）采用了 GELU 系列激活，而后续架构也采用了门控线性单元变体。通用门控单元使用两个学习的投影并将它们按元素组合：

$$\mathrm{Gate}(x) = \phi(xW_g) \odot (xW_u).$$

第三个投影将门控隐藏表示（Hidden Representation）映射回宽度 d.

Shazeer 系统地比较了 Transformer 前馈（Feed-forward）子层中的 GLU 变体，发现了几种门控变体，包括 SwiGLU 和 GEGLU，其质量比研究设置中常用的 ReLU 或 GELU 基线有所提高。 [7]

激活（Activation）选择会更改多个函数调用。门控（Gating）引入了一个额外的投影矩阵（Projection Matrix），因此改变了参数计算、计算以及用于计算等效比较的隐藏宽度选择。

研究依据：[7]、[23]

18 - 斯威格鲁机械

对应原稿第 20 页

两个向上投影，一个门控产品，一个向下投影

可以写出一个通用的 SwiGLU MLP

$$g = \mathrm{SiLU}(xW_g), \qquad u = xW_u,$$

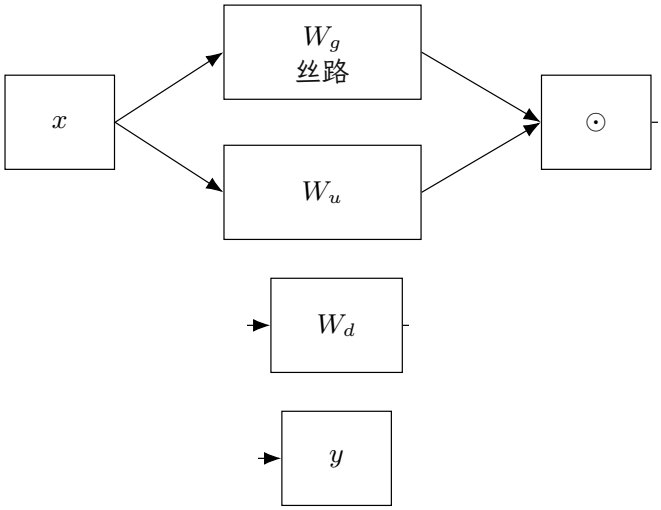
$$h = g \odot u, \qquad y = hW_d,$$

其中 $W_g, W_u \in \mathbb{R}^{d\times m}$和 $W_d \in \mathbb{R}^{m\times d}$.SiLU（也称为 Swish，$\beta$= 1）是

$$\mathrm{SiLU}(z) = z\,\sigma(z).$$

元素乘积通过另一个变换流来门控一个投影流。

LLaMA 显式地将 ReLU 替换为 SwiGLU，并减少相对于4dFFN 的隐藏宽度以说明第三个矩阵（Matrix）。[8] PaLM 同样使用 SwiGLU 并强调计算等效比较。 [10]



研究依据：[7]、[8]、[10]

19 - 为什么门控 MLPS 使用不同的宽度

对应原稿第 21 页

在比较激活之前匹配参数预算

两个矩阵（Matrix） FFN 大约有

$$P_{\text{FFN}} = 2dm.$$

三矩阵（Matrix） SwiGLU 型 FFN 大约有

$$P_{\text{SwiGLU}} = 3dm.$$

如果 ReLU 基线（Baseline）使用 $m_{\text{ReLU}} = 4d$,则匹配参数计数给出

$$2d(4d) = 3d\, m_{\text{SwiGLU}},$$

so

$$m_{\text{SwiGLU}} = \frac{8}{3}d \approx 2.67d.$$

这就是将$4d$ReLU FFN 与$4d$SwiGLU FFN 进行比较并将每个差异归因于激活（Activation）会产生误导的原因：后者具有超大的投影矩阵（Projection Matrix）。

LLaMA 将其 SwiGLU 尺寸描述为$\frac{2}{3}(4d)$。 [8]

特定于架构的舍入（Rounding）之前的。 [8]

理解框架（Mental Model）。当投影矩阵的数量发生变化时，隐藏宽度必须是任何公平参数/计算比较的一部分。

研究依据：[7]、[8]、[10]

20 - 工作参数示例

对应原稿第 22 页

为什么 **MLP** 经常主导密集块的参数

拿一个说明性的块

$$d = 768, \qquad H_q = H_{kv} = 12, \qquad d_h = 64.$$

对于标准 MHA，忽略偏差，

$$P_{\text{attn}} = 4d^2 = 4(768)^2 = 2{,}359{,}296,$$

覆盖 W_Q, W_K, W_V, W_O.

ReLU FFN 和 $m = 4d = 3072$有

$$P_{\text{FFN}} = 2(768)(3072) = 4{,}718{,}592.$$

所以这两个主要分支包含大约

$$7{,}077{,}888$$

规范和偏差之前的投影参数，其中 FFN 占总数的三分之二。

现在选择一个参数匹配的SwiGLU宽度 $m = 2048 = \frac{8}{3}d$:

$$P_{\text{SwiGLU}} = 3(768)(2048) = 4{,}718{,}592.$$

门控 MLP 具有与本示例中的$4d$二矩阵（Matrix） FFN 完全相同的矩阵参数计数。

重要。这是一个说明性的密集 MHA 块。 GQA/MQA 减少 K/V 投影参数，MoE 更改总计与活动参数计算。

研究依据：[1]、[7]、[8]、[11]

21 - 加工形状追踪

对应原稿第 23 页

一个块，端到端

采取

$$B = 2, \quad T = 4, \quad d = 8, \quad H_q = H_{kv} = 2, \quad d_h = 4, \quad m = 32.$$

剩余输入为

$$X : [2, 4, 8].$$

归一化（Normalization）使$[2, 4, 8]$.Q/K/V 重塑为

$$[2, 2, 4, 4].$$

对于自注意力（Self-attention），每个头的分数张量（Tensor）具有形状

$$[2, 2, 4, 4],$$

因为 4 个查询（Query）位置中的每一个都得分 4 个关键位置。经过值聚合和头连接后，注意力机制（Attention）分支返回

$$[2, 4, 8].$$

因此，第一个残余添加仍然是$[2, 4, 8]$.

MLP 独立扩展每个词元（Token）：

$$[2, 4, 8] \to [2, 4, 32] \to [2, 4, 8].$$

第二个残差添加再次将块输出保留在

$$[2, 4, 8].$$

理解框架（Mental Model）。通过普通块，序列长度仍为 T。注意力机制（Attention）改变了位置交互的方式，而剩余宽度在每个分支之后返回到 d。

研究依据：[1]

22 - 串行与并行剩余块

对应原稿第 24 页

Attention 和 MLP 不需要严格顺序

熟悉的 Pre-Norm 块是串行的：MLP 在注意力机制（Attention）更新后接收残差状态。以简化形式来说，

$$u = x + \mathrm{Attn}(\mathrm{Norm}(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}(u)).$$

PaLM 使用并行公式，其中注意力机制（Attention）和 MLP 读取相同的归一化残差输入，并将它们的输出加在一起：[10]

$$y = x + \mathrm{Attn}(\mathrm{Norm}(x)) + \mathrm{MLP}(\mathrm{Norm}(x)).$$

PaLM 论文报告称，这使得注意力机制（Attention）和 MLP 输入矩阵（Matrix）乘法得以融合，并在其设置中将大规模训练（Training）速度提高了约 15%，同时其消融显示出与规模相关的质量效应。 [10]

重要。并行残差块在代数上与串行块不同。 MLP 在生成自己的更新之前不再看到当前块的注意力机制（Attention）更新。

研究主播：[10]

23 - 编码器（Encoder）和解码器（Decoder）模块系列

对应原稿第 25 页

子层取决于模型（Model）系列

2017 Transformer 使用两种不同的堆栈。 [1]

积木家族	自我关注	交叉注意力	MLP
Encoder	双向/全因果 因果	no	yes
Decoder-only LM		no	yes
Encoder-decoder decoder		yes, over encoder output	yes

仅编码器（Encoder）解码器（Decoder） LM 编码器-解码器解码器

是的，通过编码器（Encoder）输出

多层线性规划

是的，是的，是的

交叉注意力（Cross-attention）添加了另一个残差子层，因为它的查询（Queries）来自解码器（Decoder）状态，而它的键/值来自编码器（Encoder）表示。仅解码器 LLM 完全省略了交叉注意子层。

现代多模态系统可以重新引入交叉注意力（Cross-attention）或其他特定模态的块，但“Transformer 块（Transformer Block）”应始终相对于正在讨论的架构进行解释。

理解框架（Mental Model）。自注意力（Self-attention）描述了 Q/K/V 从何而来。因果关系描述了哪些位置可以相互作用。交叉注意力（Cross-attention）描述了 K/V 的单独来源。

研究依据：[1]

24 - 因果掩盖受到关注

对应原稿第 26 页

块拓扑和连通性是不同的问题

仅解码器（Decoder）块通常称为“因果 Transformer 块（Transformer Block）”，但因果关系是在注意力机制（Attention）计算内部强制执行的。对于查询（Query）位置 i和关键位置 j,，可以使用附加掩模

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

然后注意力机制（Attention）计算

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V.$$

周围的残差/MLP 拓扑本身不会成为因果关系。 MLP 是位置式的，因此它不会产生跨位置泄漏，但注意子层必须强制执行允许的连接。

当前的 PyTorch 缩放点积注意力（Dot-product Attention）将因果行为暴露为注意力机制（Attention）操作，并记录非方形情况的单独查询（Query）和源长度。 [22]

重要。“解码器（Decoder）块”并不是“无处不在的下三角矩阵（Matrix）”的同义词。因果限制属于查询（Query）和关键位置之间的注意关系。

研究依据：[1]、[22]

25 - MOE 取代 MLP 分支

对应原稿第 27 页

稀疏专家（**Experts**）改变条件计算，而不是块骨架

在许多稀疏专家（Experts）混合变压器中，密集的 MLP 子层被路由专家库取代。对于词元（Token）状态 x,，路由器（Router）选择一小组专家并组合他们的输出：

$$y_{\text{MoE}} = \sum_{i \in \text{顶部-}k} \tilde{p}_i(x) E_i(x).$$

MoE 分支仍然必须返回宽度为 d的向量（Vector），以便残差更新保持有效：

$$x \leftarrow x + \text{MoE}(\text{Norm}(x)).$$

这就是为什么 MoE 可以理解为前馈（Feed-forward）分支的改变，而不是完全不同的 Transformer 拓扑。

例如，Qwen3 的 MoE 模型（Models）围绕注意力机制（Attention）/规范共享相同的基本密集模型架构，但用细粒度路由专家（Experts）取代了前馈（Feed-forward）计算。 [17]

重要。 MoE 稀疏性和注意力机制（Attention）稀疏性是不同的。 MoE 稀疏化词元（Token）的哪些前馈（Feed-forward）参数处于活动状态；稀疏注意力（Sparse Attention）会改变词元对交互的情况。

研究依据：[17]、[24]

26 - QK-NORM 不是块标准化

对应原稿第 28 页

标准化也可以出现在注意力机制（Attention）分支内

Pre-Norm 或 Post-LN 描述了范数相对于剩余分支的位置。现代模型（Models）还可以标准化内部注意力机制（Attention）张量。

Qwen3 报告 RMSNorm 在块级别具有预归一化（Normalization），并且在注意力机制（Attention）中单独使用 QK-Norm 来稳定训练（Training）。 [17] 这些是不同的标准化站点：

• 块归一化：在子层之前或之后变换残差流输入；

• QK-Norm：在分数计算之前规范化关注内的查询（Query）/键表示。

移动或移除其中一个不能被视为是另一个。

理解框架（Mental Model）。模型（Model）可以包含多个具有不同张量、轴、目的和参数的归一化。“Uses RMSNorm”不足以重建块。

研究依据：[17]

27 - GEMMA 2：一个有用的反例

对应原稿第 29 页

现代区块并不都收敛于一种规范布局

人们很容易将当代 LLM 区块概括为“Pre-RMSNorm + 注意力机制（Attention） + SwiGLU”。 Gemma 2 是不普遍化该模式的一个有用的理由。

其技术报告用 RMSNorm 指定了前规范和后规范：每个注意力机制（Attention）和前馈（Feed-forward）子层的输入和输出都被规范化。它还使用 GeGLU 而不是 SwiGLU、GQA 和交替的局部/全局注意力（Global Attention）层。 [16]

这意味着 Gemma 2 风格的子层在概念上更接近

$$y = x + \mathrm{PostNorm}(F(\mathrm{PreNorm}(x))),$$

具有由模型（Model）确定的确切实现细节。

重要。“变压器”这个词代表的是一个系列，而不是一张冻结的 2017 年接线图。准确的解释应该说明正在描述哪种架构。

研究依据：[16]

28 - 训练（Training）、预填充（Prefill）和解码（Decode）

对应原稿第 30 页

相同的块参数，截然不同的执行形状

当模型（Model）开始生成词元（Tokens）时，不会切换到不同的数学模块。改变的是作品的形态以及什么状态可以重用。

	Training/full sequence	Prompt prefill	Incremental decode
Query positions	many	many	usually one/few new positions
Past K/V	computed in pass	build cache	read from cache
MLP positions	many in parallel	many in parallel	new position(s)
Sequence dependency	causal mask for LM	causal mask	history already cached

查询（Query）位置

过去的 K/V MLP 位置 序列依赖性

训练（Training）/全序列

许多

并行计算 LM 的多个并行因果掩码（Causal Mask）

及时预填

许多

并行构建多个缓存因果掩码（Causal Mask）

增量解码（Decode）

通常一个/几个新位置从缓存中读取 新位置历史记录已缓存

在解码（Decode）期间，最新的词元（Token）仍然通过每个 Transformer 块（Transformer Block）。在每个注意力机制（Attention）子层，它的查询（Query）关注缓存的 K/V 历史记录。 MLP 保持位置明智，只需要处理该解码步骤的新词元状态。

理解框架（Mental Model）。 KV 缓存改变了注意力机制（Attention）分支重用历史记录的方式。它不会跳过新生成的词元（Token）的块的其余部分。

研究依据：[4]、[22]

29 - 键值缓存（KV Cache）属于注意力机制（Attention）状态

对应原稿第 31 页

在解码（Decode）步骤中什么是持久的

对于标准因果块，可重用的每个词元（Token）历史记录是注意力机制（Attention）键/值状态（或特定于架构的压缩等效项），而不是整个块作为通用“内存”的输出。

在解码（Decode）步骤 t,，层计算新位置的新查询（Query）/键/值。新查询与早期位置的缓存键和值进行交互。附加新的 K/V 以供后续步骤使用。 MLP 没有类似的序列缓存，因为它是位置式的：一旦当前词元（Token）离开 MLP，该分支就不需要在未来的解码步骤中重新读取其早期的中间激活。

诸如 MLA 之类的架构确切地改变了缓存的内容，但可重用状态仍然是注意力机制（Attention）的一部分。[14]

重要。 键值缓存（KV Cache）内存随着缓存的序列状态而增长。剩余流宽度和 MLP 宽度是单独的资源问题。

研究依据：[14]、[22]

30 - 块级参数核算

对应原稿第 32 页

注意力机制（Attention）和 MLP 公式揭示了架构的权衡

忽略偏差和规范，标准 MHA 和 $H_q d_h = d$使用大约

$P_{\text{MHA}} = 4d^2$.

对于 GQA，Q 和输出矩阵各自大致保持为 d^2，而 K 和 V 仅投影到

$$H_{kv} d_h:$$

$P_{\text{GQA}} \approx 2d^2 + 2d(H_{kv} d_h)$.

对于 $d = 4096$, $H_q = 32$, $H_{kv} = 8$, $d_h = 128$:

$P_{\text{MHA}} = 67{,}108{,}864$,

$P_{\text{GQA}} = 41{,}943{,}040$.

对于这个说明性配置，这些注意力机制（Attention）投射参数减少了 37.5%。

稠密两矩阵（Matrix）$4d$FFN 将包含$8d^2$矩阵参数，而参数匹配的 SwiGLU 也接近$8d^2$.这些公式解释了为什么稠密 LLM 块通常将更多的矩阵参数投入到 MLP 而不是注意力机制（Attention）。

重要。参数计数不是运行时的。序列长度、内核（Kernel）效率、KV 流量、批处理（Batching）和硬件可以使注意力机制（Attention）或 MLP 主导不同的执行机制。

研究依据：[1]、[7]、[11]

31 - 计算直觉：注意力机制（Attention） VS MLP

对应原稿第 33 页

序列混合和特征混合规模不同

对于序列长度上的密集自注意力（Self-attention） T,，QK 和 AV 产品包含序列长度的二次项，大致如下

$$O(T^2 d)$$

当头维度聚合时。线性投影贡献的项大致成比例

$$\text{to } Td^2.$$

密集 MLP 独立地应用于每个位置，因此其主导矩阵（Matrix）乘法的缩放比例大致为

$$O(Tdm)$$

对于两个矩阵（Matrix） FFN（门控变体具有常数因子变化）。

这意味着相对瓶颈取决于形状体系。在适度的上下文长度和较大的模型（Model）宽度下，MLP/投影 GEMM 可能是工作的主要部分。在长密集上下文中，注意力机制（Attention）的成对序列项快速增长。在单词元（Token）解码过程中，注意力机制面临着不断增长的 KV 读取，而 MLP 继续仅处理当前词元。

理解框架（Mental Model）。不要问“注意力机制（Attention）更贵还是 MLP 更贵？”无需指定序列长度、批次、宽度、架构、相位、精度（Precision）和硬件。

研究依据：[1]、[22]

32 - 激活（Activation）内存和重新计算

对应原稿第 34 页

训练（Training）存储的不仅仅是参数

在训练（Training）期间，反向传播需要来自前向传播（Forward Pass）的信息。因此，除了权重、梯度（Gradient）和优化器状态内存之外，变压器块还会消耗大量激活（Activation）内存。

简单的实现可能会保留归一化输入、Q/K/V 投影、注意力机制（Attention）中间体、MLP 激活以及后向内核所需的剩余状态。融合内核和重新计算策略可以准确改变存储的内容。

激活（Activation）检查点故意保存更少的中间激活，并在向后过程中重新计算部分前向传播（Forward Pass），以额外的计算换取较低的激活内存。这是围绕块的训练（Training）系统策略，而不是对块所代表的数学函数的改变。

重要。“块内存”是含糊不清的。参数内存、训练（Training）激活、优化器状态、临时内核（Kernel）工作空间和推理（Inference） 键值缓存（KV Cache）是不同的数量。

研究依据：[25]

33 - 辍学、偏见和其他小选择

对应原稿第 35 页

配置细节并不通用 **Transformer** 法则

两种实现可以共享相同的高级框图，但在许多细节上有所不同：

• 线性预测是否包含偏差项；

• 在给定模型（Model）中，什么地方应用了dropout，以及它是否为零；

• 归一化（Normalization）epsilon；

• 嵌入向量（Embeddings）/输出权重是否绑定；

• 精确的旋转尺寸和比例；

• 注意力机制（Attention）是否使用局部窗口、软上限或额外标准化；

• 激活（Activation）和门控MLP 约定；

• 张量（Tensor）并行或融合内核（Kernel）布局。

这些选择会影响检查点兼容性、参数计数、数值行为和内核（Kernel）选择。它们应该从模型（Model）的论文/配置/代码中读取，而不是从“Transformer”一词中推断出来。

Gemma 2 的 logit soft-capping 和 Qwen3 的 QK-Norm 是架构细节的示例，这些细节很容易从通用图中省略，但对精确的实现很重要。 [16,17]

研究依据：[16]、[17]

34 - 透明的 Pytorch 块

对应原稿第 36 页

教学预 RMSNorm 关注 + SwiGLU

以下可执行草图公开了操作顺序。它使用普通的 MHA 并故意省略 GQA、RoPE、KV 缓存、融合内核、张量（Tensor）等生产问题

并行性、丢失和特定于模型的偏差。

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        assert d % n_heads == 0
        self.h, self.dh = n_heads, d // n_heads
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.out = nn.Linear(d, d, bias=False)

    def forward(self, x):
        B, T, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        def heads(z):
            return z.view(B, T, self.h, self.dh).transpose(1, 2)
        q, k, v = map(heads, (q, k, v))
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        y = y.transpose(1, 2).reshape(B, T, D)
        return self.out(y)

class Block(nn.Module):
    def __init__(self, d, n_heads, mlp_dim):
        super().__init__()
        self.norm1, self.norm2 = nn.RMSNorm(d), nn.RMSNorm(d)
        self.attn = CausalSelfAttention(d, n_heads)
        self.gate = nn.Linear(d, mlp_dim, bias=False)
        self.up = nn.Linear(d, mlp_dim, bias=False)
        self.down = nn.Linear(mlp_dim, d, bias=False)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        h = self.norm2(x)
        h = F.silu(self.gate(h)) * self.up(h)
        return x + self.down(h)
```

这与串行 Pre-Norm 残差拓扑和三投影 SwiGLU 结构相匹配。一个

真实模型（Model）必须与其精确的注意力头布局、RoPE 规则、范数 epsilon、MLP 维度、缓存语义和检查点参数名称相匹配。

重要。教学代码不是 LLaMA、Qwen、Gemma 或其他模型（Model）的直接复制。精确的架构是一个兼容性契约。

研究依据：[6]、[7]、[8]、[17]、[20]、[22]

35 - 常见的实施错误

对应原稿第 37 页

大多数块错误是顺序、形状或契约错误

1. 将 LN 后方程用于 LN 前检查点。相同的模块，不同的功能。

2. 标准化错误的张量（Tensor）。块范数、QK-范数和后子层范数是不同的站点。

3.忘记注意力机制（Attention）输出投射。头连接不是完整的分支。

4. 默认将RoPE应用于残差流（Residual Stream）或V。标准 LLaMA 式 RoPE 目标为 Q/K。

5. 假设 $H_q = H_{kv}$·GQA/MQA 打破了这个等式。

6. 对 SwiGLU 使用二矩阵（Matrix） FFN 参数公式。门控（Gating）增加了第三个大投影。

7. 添加宽度不匹配的张量。每个剩余分支必须返回 d.

8. 将因果掩码视为块级操作。它属于注意力机制（Attention）连接内部。

9. 复制通用 MLP 宽度。扩展比率是特定于架构的。

10. 将工作张量形状（Tensor Shape）与检查点兼容性等同起来。 Epsilon、偏差、捆绑、排序和约定仍然很重要。

研究依据：[1]、[8]、[11]、[12]、[16]、[17]

36 - 常见的误解

对应原稿第 38 页

创造错误思维模式的捷径

误解	修正
“注意力机制输出取代了词元状态。”	它通常作为剩余更新添加。

“MLP 混合了词元（Tokens）。”它按位置行事；上下文可能已经存在于每个状态中。“RMSNorm 就是 LayerNorm，没有偏见。” RMSNorm 还省略了均值减法并使用不同的尺度统计量。“LN 之前和 LN 后是等效的重排序。”他们定义了不同的计算/梯度（Gradient）路径。“RoPE 修改了词元嵌入向量（Embeddings）。”标准 RoPE 在注意力机制（Attention）内旋转 Q/K。“每个现代 LLM 都是 Pre-RMSNorm + SwiGLU。”

Gemma 2、PaLM、Qwen3 等说明了重要的变体。“GQA 改变了整个区块。”它改变了注意力机制（Attention）的Q/K/V头部结构。“MoE 表示不同的变形金刚骨架。”它通常用路由专家（Experts）代替前馈（Feed-forward）分支。“键值缓存（KV Cache）存储整个块状态。”它存储注意力机制 K/V 或特定于架构的等效项。“注意力机制总是主导计算。”瓶颈取决于形状、阶段、架构和硬件。

研究依据：[1]、[6]、[8]、[10]、[11]、[14]、[16]、[17]

37 - 完整的理解框架（Mental Model）

对应原稿第 39 页

一条链中的 Transformer 块（Transformer Block）

现代 Transformer 块（Transformer Block）最容易记住的是九个相互关联的想法。

1.携带残留状态。每个词元（Token）都有一个宽度d向量（Vector），该向量在堆栈中持续存在。

2. 在指定位置进行标准化。 LayerNorm/RMSNorm 放置是架构的一部分。

3. 将词元（Tokens）与注意力机制（Attention）混合在一起。 Q/K/V 投影、位置处理、掩蔽和头部布局决定了位置如何相互作用。

4. 投影回 d.注意力机制（Attention）分支必须返回到剩余宽度。

5.添加更新。残差加法保留了旧的表示（Representation），同时纳入了注意力机制（Attention）的贡献。

6. 使用 MLP 变换特征。每个词元（Token）都被扩展、非线性门控/激活，并独立地投射回来。

7. 再次添加。 MLP 贡献了另一个剩余更新。

8. 对特定于架构的变体重复此操作。 GQA、QK-Norm、GeGLU/SwiGLU、局部注意力（Local Attention）、后范数、MoE 和其他选择会改变内部结构而不擦除核心逻辑。

9. 按阶段解释执行。训练（Training）、预填充（Prefill）和解码（Decode）在不同的形状和重用模式下运行相同的学习块。

理解框架（Mental Model）。持久的抽象不是“MLP 后的注意力机制（Attention）”。它是通过词元（Token）混合和特征混合计算反复更新的残余状态，其归一化（Normalization）、路由（Routing）和注意细节是特定于架构的。

研究依据：[1]、[4]、[6]、[7]、[8]、[10]、[16]、[17]

38 - 紧凑架构比较

对应原稿第 40 页

具有代表性的区块，并非普遍的进展

模型（Model）/家庭 规范安置 MLP 位置/注意力机制（Attention） 有用的课程

正弦 + MHA 原始参考块 GPT-2 Pre-LN + 最终范数 GELU 式因果 MHA 预归一化（Normalization）移至子块输入 PaLM Pre-LN，并行分支

$$\text{ReLU},\ 4d$$

变形金刚 (2017) 后 LN

SwiGLU MQA 不需要连续订购 LLaMA Pre-RMSNorm SwiGLU RoPE + MHA 常见现代致密配方 Gemma 2 前 + 后 RM- SNorm

GeGLU GQA，本地/全球现代规范放置仍然不同 Qwen3 Pre-RMSNorm + QK-Norm

SwiGLU RoPE + GQA 内部注意力机制（Attention）规范可以与块规范共存

这张表是有意选择的。它说明了为什么短语“Transformer block”应该触发一个清单而不是一个记忆的绘图：范数放置、分支排序、注意力头（Attention Head）结构、位置方法、MLP 门控（Gating）和内部注意力机制（Attention）修改都可以改变。

重要。模型（Model）系列随着版本和参数规模的变化而发展。为了准确再现，请使用相应的技术报告和发布的配置，而不是此比较表。

研究依据：[1]、[4]、[8]、[10]、[16]、[17]

39 - 参考文献和扩展阅读

对应原稿第 41 页

1. A. Vaswani 等人。您所需要的就是关注。 NeurIPS 2017.arXiv:1706.03762.

2. 何坤，张晓，任书，孙杰。用于图像识别的深度残差学习。 CVPR 2016.arXiv:1512.03385.

3. J. L. Ba、J. R. Kiros、G. E. Hinton。层标准化。 arXiv:1607.06450，2016。

4.A.雷德福等人。语言模型（Models）是无监督的多任务学习器。 OpenAI 技术报告，2019 年。

5.R.熊等人。关于 Transformer 架构中的层标准化。 ICML 2020.arXiv:2002.04745.

6. B. 张，R. Sennrich。均方根层归一化（Layer Normalization）。 NeurIPS 2019.arXiv:1910.07467.

7.N.沙泽尔。 GLU 变体改进了 Transformer。 arXiv:2002.05202，2020。

8. H.Touvron 等人。 LLaMA：开放且高效的基础语言模型（Models）。 arXiv:2302.13971，2023。

9.H.Touvron 等人。 Llama 2：开放基础和微调聊天模型（Models）。 arXiv:2307.09288，2023。

10. A.乔杜里等人。 PaLM：通过路径扩展语言建模。 JMLR 2023 年 24 月。

11.J.安斯利等人。 GQA：从多头检查点训练（Training）通用多查询（Query）变压器模型（Models）。 EMNLP 2023.arXiv:2305.13245.

12. J.苏等人。 RoFormer：具有旋转位置嵌入（Rotary Position Embedding）的增强型变压器。 arXiv:2104.09864，2021。

13. T. Dao 等人。 FlashAttention：具有 IO 感知能力的快速、内存高效的精确注意力（Exact Attention）。 NeurIPS 2022.arXiv:2205.14135.

14. DeepSeek-AI 等人。 DeepSeek-V2：强大、经济且高效的专家混合语言模型（Model）。 arXiv:2405.04434，2024。

15.A.Jiang 等人。专家（Experts）的混合。 arXiv:2401.04088，2024。

16.杰玛团队。 Gemma 2：以实用的规模改进开放语言模型（Models）。 arXiv:2408.00118，2024。

17.A.杨等人。 Qwen3 技术报告。 arXiv:2505.09388，2025。

18.T.布朗等人。语言模型（Models）是少样本学习者。 NeurIPS 2020.arXiv:2005.14165.

19.PyTorch 文档。 torch.nn.LayerNorm。当前文档。

20.PyTorch 文档。 torch.nn.RMSNorm。当前文档。

21. H. Wang 等人。 DeepNet：将 Transformer 扩展到 1,000 层。 arXiv:2203.00555，2022。

22.PyTorch 文档。 torch.nn.function.scaled_dot_product_attention。当前文档。

23.D.亨德里克斯，K.金佩尔。高斯误差线性单位 (GELU)。 arXiv:1606.08415，2016。

24.N.Shazeer 等人。极其庞大的神经网络：稀疏门控专家混合层。 ICLR 2017.arXiv:1701.06538.

25. T. Chen 等人。使用次线性内存成本训练（Training）深度网络。 arXiv:1604.06174，2016。